

Redes Neurais Artificiais - Instância: 2025.2 - Prof. Dr. Ryan Azevedo
Grupo: José Daniel, Rian Wilker e Pedro Medeiros
Time: Chicago Bulls

Justificativa das Features Utilizadas

O modelo foi treinado utilizando um conjunto específico de features, que foram selecionadas com base em sua relevância e poder preditivo demonstrados na Atividade 1 (Regressão Linear e Logística).

As **7 features** escolhidas foram:

1. **PTS** (Pontos)
2. **FG_PCT** (Percentual de Arremessos de Quadra)
3. **FG3_PCT** (Percentual de Arremessos de 3 Pontos)
4. **REB** (Rebotes)
5. **AST** (Assistências)
6. **STL** (Roubos de Bola)
7. **TOV** (Turnovers - Perdas de Bola)

Qual a configuração/arquitetura da MLP implementada:

A arquitetura da rede foi definida no método `build_model` do script.

• Qual Função de Ativação utilizada?

Foram utilizadas duas funções de ativação distintas:

- **Camadas Ocultas: ReLU (Rectified Linear Unit)**
 - **Onde:** `activation='relu'`
 - **Justificativa:** É a função de ativação padrão da indústria para camadas ocultas. Ela é computacionalmente eficiente e ajuda a mitigar o "problema do desaparecimento do gradiente" (vanishing gradient), permitindo um treinamento mais rápido e eficaz de redes mais profundas.
- **Camada de Saída: Sigmoid**
 - **Onde:** `self.model.add(Dense(1, activation='sigmoid', ...))`
 - **Justificativa:** Como o problema é uma **classificação binária** (Vitória=1, Derrota=0), a função Sigmoid é essencial. Ela comprime qualquer valor de entrada para um intervalo entre 0 e 1, que pode ser diretamente interpretado como a **probabilidade** da equipe vencer o jogo.

- **Quantas camadas ocultas utilizou?**

2 (duas) camadas ocultas.

- **Onde:** `hidden_layers=[32, 16]` (Linha 144)
- **Justificativa:** Duas camadas ocultas oferecem um bom equilíbrio entre capacidade de aprendizado e complexidade. Isso permite que a rede aprenda relações não-lineares complexas entre as features de entrada (primeira camada) e, em seguida, combine essas representações aprendidas em padrões de nível superior (segunda camada) antes de tomar a decisão final.

- **Quantos neurônios na camada de Entrada, Oculta e Saída?**

Camada de Entrada: 7 neurônios

- **Onde:** `input_dim = self.X_train.shape[1]` (Linha 149) e `features = [...]` (Linha 722)
- **Justificativa:** O número de neurônios na camada de entrada é **sempre** igual ao número de features. Como utilizamos 7 features (PTS, REB, AST, etc.), a camada de entrada possui 7 neurônios.

Camadas Ocultas: 32 e 16 neurônios

- **Onde:** `hidden_layers=[32, 16]` (Linha 144)
- **Justificativa:** A rede usa uma arquitetura "funil":
 - **1ª Camada Oculta:** 32 neurônios.
 - **2ª Camada Oculta:** 16 neurônios.
- Essa abordagem é comum, pois a primeira camada (mais larga) captura uma variedade de interações das features, e a segunda camada (mais estreita) força a rede a comprimir essa informação, aprendendo uma representação mais densa e saliente dos dados.

Camada de Saída: 1 neurônio

- **Onde:** `self.model.add(Dense(1, activation='sigmoid', ...))` (Linha 172)
- **Justificativa:** Para classificação binária, um único neurônio com ativação Sigmoid é o método padrão e mais eficiente. Seu valor de saída (entre 0 e 1) representa a probabilidade da classe "Vitória".

• Quantas épocas?

Um máximo de 200 épocas.

- **Onde:** `epochs=200` (Linha 192)
- **Justificativa:** 200 é o número *máximo* de iterações definidas para o treinamento. No entanto, o número *real* de épocas executadas foi controlado pelo **Early Stopping** (paciência de 20 épocas), que interrompe o treinamento automaticamente se o desempenho no conjunto de validação não melhorar, evitando overfitting.

O processo de treinamento e avaliação foi gerenciado pelos métodos `train_model` e `evaluate_model`.

• Usou Dense, Dropout, BatchNormalization, optimizer, loss?

Sim, todos foram utilizados.

- **Dense:** Sim, é a camada fundamental da MLP (totalmente conectada).
- **Dropout:** Sim, com uma taxa de 30% (`dropout_rate=0.3`), para prevenir overfitting.
- **BatchNormalization:** Sim, utilizada após cada camada Densa (antes do Dropout) para estabilizar e acelerar o treinamento.
- **Optimizer:** Sim, a implementação testa três otimizadores diferentes: **SGD**, **Adam** e **RMSprop**.
- **Loss (Função de Perda):** Sim, foi utilizada a '`binary_crossentropy`'. Esta é a função de perda matematicamente correta para um problema de classificação binária com saída Sigmoid.

• Porcentagem de treinamento e validação?

A divisão dos dados, definida no método `prepare_data` (Linha 92), foi:

- 70% para Treinamento
- 15% para Validação
- 15% para Teste

Justificativa: O script primeiro separa 15% para teste (`test_size=0.15`). Dos 85% restantes, ele aloca uma fração (`val_size_adjusted`) que resulta em 15% do total para validação, deixando os 70% restantes para o treinamento.

• Treinou com early stopping e validation split?

Sim.

- **Early Stopping:** (Linha 226) Foi explicitamente configurado (`early_stopping=True`) para monitorar a perda de validação (`monitor='val_loss'`).
- **Patience (Paciência):** 20 épocas (`patience=20`).
- **Validation Split:** (Linha 238) O conjunto de validação (`self.X_val`, `self.y_val`) foi passado diretamente para o método `.fit()`, garantindo que o Early Stopping monitorasse dados que o modelo não via durante o treino.
- **Restore Best Weights:** (Linha 230) A opção `restore_best_weights=True` foi ativada, garantindo que o modelo final retornado fosse aquele com o menor erro de validação, e não o modelo da última época.

• Calculou MAE, RMSE e R2 para cada estatística?

Sim.

- **Onde:** Método `evaluate_model` (Linhas 285-287).
- **Justificativa:** Embora sejam métricas de regressão, elas foram calculadas sobre as **probabilidades** de saída (contínuas de 0 a 1) em relação aos valores reais (0 ou 1). Isso fornece uma medida de quão "confiante" e "preciso" o modelo está em suas previsões de probabilidade.
 - **MAE (Mean Absolute Error)**
 - **RMSE (Root Mean Squared Error)**
 - **R2 (R-squared)**

• O que a equipe fez na implementação para evitar overfitting?

Utilizamos três técnicas principais para combater o overfitting:

1. **Dropout (30%):** Durante o treinamento, 30% dos neurônios de cada camada oculta eram "desligados" aleatoriamente em cada passo. Isso força a rede a aprender de forma mais robusta, impedindo que neurônios específicos se tornem co-dependentes.
2. **Early Stopping:** O treinamento foi monitorado no conjunto de validação (15% dos dados). Se a perda de validação não melhorasse por 20 épocas seguidas, o treinamento era interrompido.
3. **Batch Normalization:** Embora sua função principal seja acelerar o treinamento, a Batch Normalization adiciona um leve ruído a cada lote de

dados, o que age como um leve regularizador, ajudando a prevenir o overfitting.

- **Testem nas implementações o uso de Stochastic Gradient Descent (SGD), Adam (Adaptive Moment Estimation) e RMSProp.**

Sim, os três foram testados e comparados.

- **Onde:** A função `comparar_otimizadores` foi projetada especificamente para essa finalidade.
- **Execução:** A função itera sobre uma lista `optimizers = ['sgd', 'adam', 'rmsprop']`, treina, avalia e gera todos os gráficos e relatórios para cada um dos três otimizadores, permitindo uma comparação direta de desempenho. Os resultados foram compilados em `comparacao_otimizadores.csv`.